

Notes on Computer Vision

Sebastian Ruiz, updates by Julian Kerl

29 September 2023, updated 7 November 2024

1 Aruco Detection with OpenCV

We calibrated the camera and we have provided you with `camera_matrix` and `dist_coeffs`. You can use those for undistorting the image, or directly pass them to the `solvePnP` function for single marker pose estimation.

2 Rotations

2.1 Rvec and Tvec

The tvec of a marker is the translation (x, y, z) of the marker from the origin.

The rvec of a marker is an axis-angle rotation^{1 2}. You can use the `rotation_matrix` function in notebook 2 to convert them to a rotation matrix. Bare in mind that this rotation matrix performs *intrinsic rotation*!

2.2 Rotation Matrix

A rotation matrix is a transformation matrix that is used to perform a rotation in Euclidean space.

In the 2D case the matrix

$$R = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}$$

rotates points in the xy plane counterclockwise through an angle θ . Namely, for a vector $\mathbf{v}$

$$R\mathbf{v} = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} x \cos \theta - y \sin \theta \\ x \sin \theta + y \cos \theta \end{bmatrix}.$$

The 3D case is similar.

2.3 Euler Angles

Euler angles are a sequence of 3 rotations around a sequence of axes. The axes of the original frame are denoted as x, y, z and the axes of the rotated frame as X, Y, Z .

The three elemental rotations may be **extrinsic**. That is to say, the rotations apply about the axes xyz of the original, fixed, coordinate system.

Alternatively, the three elemental rotations may be **intrinsic**. That is to say, the rotations apply about the axes of the rotating coordinate system XYZ . After applying the first rotation about the first axis, the second rotation is performed on the new coordinate system.

The `rotation_matrix` function in notebook 2 performs intrinsic rotations. So:

¹https://en.wikipedia.org/wiki/Axis%E2%80%93angle_representation

²<https://stackoverflow.com/questions/12933284/rodrigues-into-eulerangles-and-vice-versa#12977143>

```

1 x_rotation = np.radians(-135)
2 y_rotation = np.radians(45)
3 z_rotation = np.radians(90)
4 R = rotation_matrix(x_rotation, y_rotation, z_rotation)

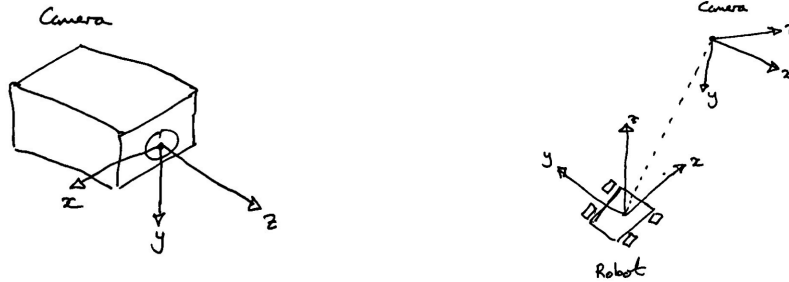
```

will first rotate 90° around the original z axis, then 45° around the resulting y axis, and finally -135° around the then resulting x axis.

3 Rigid-Body Transforms

We use the right-hand coordinate system.

Objects seen in the camera have position with respect to the camera coordinate system. The coordinate system is defined as shown below. We position the camera on the robot. We define the robot coordinate system such that x is forward, and z is up.

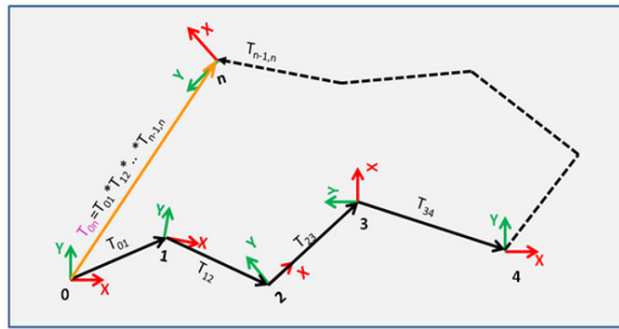


We can represent position and rotation in matrix form. This matrix is called the (rigid body) transform matrix.

$$T = \left[\begin{array}{ccc|c} R & & & t \\ \hline 0 & 0 & 0 & 1 \end{array} \right]$$

where R is a 3×3 rotation matrix and t is a translation vector of length 3.

We can chain transform matrices.



source:

[rigid-body-transformations](https://www.rosroboticslearning.com/rigid-body-transformations)

<https://www.rosroboticslearning.com/>

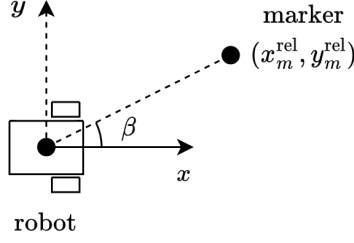
here

$$T_{0,n} = T_{0,1} \cdot T_{1,2} \cdots T_{n-1,n}$$

If the marker transform w.r.t. the camera is given by $T_{\text{camera,marker}}$ and the camera transform w.r.t. the robot base is given by $T_{\text{robot,camera}}$, then

$$T_{\text{robot,marker}} = T_{\text{robot,camera}} \cdot T_{\text{camera,marker}}$$

We can therefore find the marker position w.r.t. the robot, $(x_m^{\text{rel}}, y_m^{\text{rel}})$.



Robot Coordinate system, with marker at position $(x_m^{\text{rel}}, y_m^{\text{rel}})$ with angle β . In real world coordinates we denote the marker position as (x_m, y_m) .

4 Camera Intrinsics and Extrinsics

We have the following equation, such that given a point in world space $(X_w, Y_w, Z_w, 1)^\top$, we can get the point on the image $(u, v, w)^\top$ in homogeneous coordinates,

$$\begin{pmatrix} u \\ v \\ w \end{pmatrix} = \underbrace{\begin{pmatrix} f_u & s & p_u & 0 \\ 0 & f_v & p_v & 0 \\ 0 & 0 & 1 & 0 \end{pmatrix}}_{=K} \underbrace{\begin{pmatrix} R & | & t \\ \hline 0 & 0 & 0 & 1 \end{pmatrix}}_{=Q} \begin{pmatrix} X_w \\ Y_w \\ Z_w \\ 1 \end{pmatrix}.$$

In the above, K is called the camera intrinsic matrix, and Q is called the camera extrinsic matrix. The values f_u, f_v correspond to the focal-length and p_u, p_v correspond to the principal point, in the pinhole camera model. The extrinsic matrix is a rigid body transform from world to camera.

Let $P = KQ$. Then

$$\begin{pmatrix} u \\ v \\ w \end{pmatrix} = P \begin{pmatrix} X_w \\ Y_w \\ Z_w \\ 1 \end{pmatrix}.$$

Given a homogeneous image coordinate $(u, v, w)^\top$, the image coordinate is given by $(u/w, v/w)$ if $w \neq 0$, and if $w = 0$ the point is at infinity.

4.1 Finding the point in world coordinates given a point on the image

We follow section 6.2.2 of Harley and Zisserman's Multiple View Geometry In Computer Vision.

Write $P = [M | \mathbf{p}_4]$ where M is a 3×3 matrix and $\mathbf{p}_4$ is a column vector of length 3.

The inhomogeneous coordinates of the camera center are $\tilde{\mathbf{C}} = -M^{-1}\mathbf{p}_4$.

Let $\mathbf{x}$ be the homogeneous coordinate vector of a point on the image. A point on the image back-projects to a ray in the scene. The back projected ray for $\mathbf{x}$ goes through $\tilde{\mathbf{C}}$.

The direction vector of the ray is $M^{-1}\mathbf{x}$, so in inhomogeneous coordinates the back-projected ray is

$$\tilde{X}(\mu) = \mu M^{-1}\mathbf{x} + \tilde{\mathbf{C}}$$

We want to find the point where $Z = 0$ on the ray. Therefore we want to find μ such that

$$\begin{pmatrix} X_1 \\ X_2 \\ 0 \end{pmatrix} = \mu M^{-1}\mathbf{x} + \tilde{\mathbf{C}}$$

Let $\hat{\mathbf{x}} = M^{-1}\mathbf{x}$. Then

$$\begin{pmatrix} X_1 \\ X_2 \\ 0 \end{pmatrix} = \mu \begin{pmatrix} \hat{x}_1 \\ \hat{x}_2 \\ \hat{x}_3 \end{pmatrix} + \begin{pmatrix} \tilde{C}_1 \\ \tilde{C}_2 \\ \tilde{C}_3 \end{pmatrix}$$

Thus

$$\mu = -\frac{\tilde{\mathbf{C}}_3}{\hat{\mathbf{x}}_3}$$

gives the point on the ray where $Z = 0$. Therefore

$$\begin{pmatrix} X_1 \\ X_2 \\ 0 \end{pmatrix} = -\frac{\tilde{\mathbf{C}}_3}{\hat{\mathbf{x}}_3} M^{-1} \mathbf{x} + \tilde{\mathbf{C}}.$$